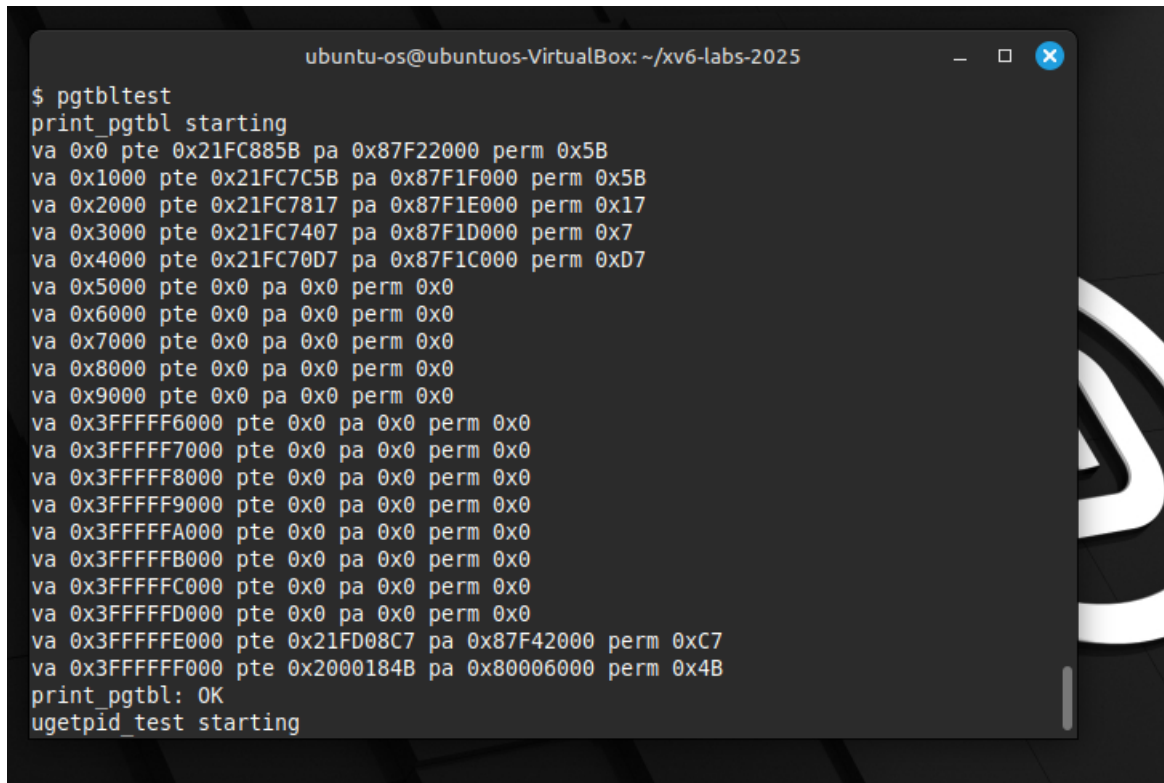


PROBLEM SET 3

→ Inspect a user-process page table (easy)

A terminal window titled 'ubuntu-os@ubuntuos-VirtualBox: ~/xv6-labs-2025' displays the output of the 'pgtbltest' command. The output lists 20 page table entries, each showing a virtual address (va), a page table entry (pte), a physical address (pa), and permission bits (perm). The first 10 entries show mappings for addresses 0x0 to 0x9000, and the last 10 entries show mappings for addresses 0x3FFFFFF000 to 0x3FFFFFFF000. The terminal also shows 'print_pgbl: OK' and 'ugetpid_test starting' at the bottom.

```
$ pgtbltest
print_pgbl starting
va 0x0 pte 0x21FC885B pa 0x87F22000 perm 0x5B
va 0x1000 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x2000 pte 0x21FC7817 pa 0x87F1E000 perm 0x17
va 0x3000 pte 0x21FC7407 pa 0x87F1D000 perm 0x7
va 0x4000 pte 0x21FC70D7 pa 0x87F1C000 perm 0xD7
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFD000 pte 0x0 pa 0x0 perm 0x0
va 0x3FFFFFFE000 pte 0x21FD08C7 pa 0x87F42000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgbl: OK
ugetpid_test starting
```

When we run `pgtbltest`, it prints the page-table entries for the first 10 and last 10 pages of a process's virtual address space.

- Va (virtual address) : what the process thinks the address is.
- Pte (page table entry) : a 64-bit number made up of the physical address and permission bits.
- Pa (physical address) : where the page actually is in memory
- Perm : Permission bits from the PTE (e.g. readable, writable, executable)

Each Page Table Entry (PTE) has a virtual address, page table entry (PTE) in hex, physical address, and permission bits.

These define how each virtual address maps to physical memory and what kind of access (read, write, execute) is allowed.

First 10 Page Table Entries

1. This is a **user page**, it is **readable and executable but not writable**. The accessed bit is set — likely a **code page**.
2. This is a **user page**, also **readable and executable but not writable**. The accessed bit is set — another **code page**.
3. This is a **user page, readable and writable but not executable** — possibly **user data**.
4. This is a **kernel page, readable and writable but not executable** — could be **kernel data**.
5. This is a **user page, readable and writable but not executable**, and both the accessed and dirty bits are set — this page has been read and written, likely part of the **user heap**.
- 6–10. These pages are **unmapped**, meaning they are not currently in use and have no valid physical address.

Last 10 Page Table Entries

- 1–8. These are **unmapped** pages (no valid mapping in the page table).
9. This is a **kernel page, readable and writable**, with accessed and dirty bits set — most likely a **kernel stack**.
10. This is a **kernel page, readable and executable but not writable** — likely **kernel code** (for example, the trampoline page).

➔ Speed up system calls ([easy](#))

- `uptime()`, As the kernel can periodically update the current tick count using the shared `USYSCALL` and the user space can read it directly without syscall
- `time()`, As the kernel periodically writes time to the shared memory

➔ Print a page table ([easy](#))

```

ubuntu-os@ubuntuos-VirtualBox: ~/xv6-labs-2025
va 0x3FFFFFFD000 pte 0x21FD4813 pa 0x87F52000 perm 0x13
va 0x3FFFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0x3FFFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000087f22000
..0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000
.. ..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
.. .. ..0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..0x00000000000001000: pte 0x0000000021fc705b pa 0x0000000087f1c000
.. .. ..0x00000000000002000: pte 0x0000000021fc6cd7 pa 0x0000000087f1b000
.. .. ..0x00000000000003000: pte 0x0000000021fc6807 pa 0x0000000087f1a000
.. .. ..0x00000000000004000: pte 0x0000000021fc64d7 pa 0x0000000087f19000
..0x00000003fc000000: pte 0x0000000021fc8401 pa 0x0000000087f21000
.. ..0x00000003fffe0000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0x00000003ffffd000: pte 0x0000000021fd4853 pa 0x0000000087f52000
.. .. ..0x00000003fffffe000: pte 0x0000000021fd00c7 pa 0x0000000087f40000
.. .. ..0x00000003fffff000: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
superpg_fork starting
pgtbltest: superpg_fork failed: pte different, pid=3
$

```

```

ubuntu-os@ubuntuos-VirtualBox: ~/xv6-labs-2025
oc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builti
n-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno
-pie -no-pie -c -o kernel/spinlock.o kernel/spinlock.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kerne
l kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kerne
l/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kerne
l/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file
.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o
kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printf.o kernel/uar
t.o kernel/spinlock.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permi
ssions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /
; /^$/d' > kernel/kernel.sym
make[1]: Leaving directory '/home/ubuntu-os/xv6-labs-2025'
== Test pgtbltest ==
$ make qemu-gdb
(13.8s)
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
== Test pgtbltest: print_kpgtbl ==
pgtbltest: print_kpgtbl: OK
== Test pgtbltest: superpg ==

```